

Stashing stashing iterators for proper flattening

Resolving LWG 3698

Document #: P2770R0
Date: 2023-01-31
Project: Programming Language C++
Audience: LWG
Reply-to: Tim Song
<t.canens.cpp@gmail.com>

§ 1 Abstract

This paper provides wording to resolve [\[LWG3698\]](#), [\[LWG3700\]](#), [\[LWG3791\]](#), and NB comment [US 61-126](#).

§ 2 Discussion

A *stashing iterator* is an iterator that returns a reference to something within itself (or more generally, *something* that is tied to the lifetime of the iterator). Such iterators can only be of the input category; forward iterators are not allowed to be stashing (25.3.4.11 [\[iterator.concept.forward\]](#) p3; 25.3.5.5 [\[forward.iterators\]](#) p6).

There are two parts to [\[LWG3698\]](#):

- `regex_iterator` and `regex_token_iterator`

These are stashing iterators that lie about their category. It is probably too breaking at this point to change their `iterator_category`, but we should add `iterator_concept` so that they give the correct answer for C++20 iterator concepts.

- `join` and `join_with`

These don't handle stashing iterators properly.

As currently specified, the `join_view` iterator holds a) an iterator `outer` into the range being joined and b) an iterator `inner` into the current element that is being iterated over (that is, `*outer`).

When `outer` is stashing, then, `inner` actually refers into `*outer`. In such a case, making a copy of the `join_view` iterator would produce an iterator that holds:

- a copy of `outer`, and
- a copy of `inner`, but this copy is pointing into the original `outer`, not the copy.

Hilarity ensues when we try to continue to iterate using this copy.

The fix is to cache the outer iterator within the view, similar to what we do for a number of other views (`lazy_split`, for instance). This way, copying the `join_view` iterator only copies the inner iterator. Because there is no dedicated “stashing” trait, we need to do this for all input iterators.

While in the vicinity, this paper also resolves [\[LWG3700\]](#) and [\[LWG3791\]](#), two relatively minor issues concerning `join_view` and `join_with_view`.

The wording below has been implemented and tested on top of `libstdc++` master.

§ 3 Wording

This wording is relative to [\[N4928\]](#).

1. Edit 32.11.1.1 [\[re.regiter.general\]](#), class template `regex_iterator` synopsis, as indicated:

```
namespace std {
    template<class BidirectionalIterator,
             class charT = typename
                 iterator_traits<BidirectionalIterator>::value_type,
             class traits = regex_traits<charT>>
    class regex_iterator {
    public:
        using regex_type      = basic_regex<charT, traits>;
        using iterator_category = forward_iterator_tag;
+       using iterator_concept = input_iterator_tag;
        using value_type      = match_results<BidirectionalIterator>;
        using difference_type  = ptrdiff_t;
        using pointer          = const value_type*;
        using reference        = const value_type&

        [...]
    };
}
```

2. Edit 32.11.2.1 [\[re.tokiter.general\]](#), class template `regex_token_iterator` synopsis, as indicated:

```
namespace std {
    template<class BidirectionalIterator,
             class charT = typename
                 iterator_traits<BidirectionalIterator>::value_type,
             class traits = regex_traits<charT>>
    class regex_token_iterator {
    public:
        using regex_type      = basic_regex<charT, traits>;
        using iterator_category = forward_iterator_tag;
+       using iterator_concept = input_iterator_tag;
        using value_type      = sub_match<BidirectionalIterator>;
        using difference_type  = ptrdiff_t;
        using pointer          = const value_type*;
        using reference        = const value_type&

        [...]
    };
}
```

```
};
}
```

3. Add the following function template to 26.7.5 [\[range.adaptor.tuple\]](#), and change the stable name to `[range.adaptor.helpers]`:

```
namespace std::ranges {
    [...]

    template<class T>
    constexpr T& as_lvalue(T&& t) {      // exposition only
        return static_cast<T&>(t);
    }
}
```

4. Edit 26.7.14.2 [\[range.join.view\]](#), class template `join_view` synopsis, as indicated:

```
namespace std::ranges {
    template<input_range V>
        requires view<V> && input_range<range_reference_t<V>>
        class join_view : public view_interface<join_view<V>> {
        private:
            using InnerRng = range_reference_t<V>;           // exposition
                only

            // 26.7.14.3 \[range.join.iterator\], class template join_view::iterator
            template<bool Const>
                struct iterator;                             // exposition
                    only

            // 26.7.14.4 \[range.join.sentinel\], class template join_view::sentinel
            template<bool Const>
                struct sentinel;                             // exposition
                    only

            V base_ = V();                                   // exposition
                only

+   non-propagating_cache<iterator_t<V>> outer_;           // exposition
+   only, present only                                   // when
+   !forward_range<V>

            non-propagating_cache<remove_cv_t<InnerRng>> inner_; // exposition
                only, present only                             // when
                !is_reference_v<InnerRng>

        public:
            join_view() requires default_initializable<V> = default;
            constexpr explicit join_view(V base);

            constexpr V base() const & requires copy_constructible<V> { return
                base_; }
            constexpr V base() && { return std::move(base_); }
```

```

    constexpr auto begin() {
+   if constexpr (forward_range<V>) {
        constexpr bool use_const = simple_view<V> &&
                                   is_reference_v<InnerRng>;
        return iterator<use_const>{*this, ranges::begin(base_)};
+   }
+   else {
+       outer_ = ranges::begin(base_);
+       return iterator<false>{*this};
+   }
    }

    constexpr auto begin() const
        requires forward_input_range<const V> &&
                 is_reference_v<range_reference_t<const V>> &&
+       input_range<range_reference_t<const V>>
    { return iterator<true>{*this, ranges::begin(base_)}; }

    constexpr auto end() {
        if constexpr (forward_range<V> &&
                     is_reference_v<InnerRng> && forward_range<InnerRng> &&
                     common_range<V> && common_range<InnerRng>)
            return iterator<simple_view<V>>{*this, ranges::end(base_)};
        else
            return sentinel<simple_view<V>>{*this};
    }

    constexpr auto end() const
        requires forward_input_range<const V> &&
                 is_reference_v<range_reference_t<const V>> &&
+       input_range<range_reference_t<const V>>
    {
        if constexpr (forward_range<const V> &&
                     forward_range<range_reference_t<const V>> &&
                     common_range<const V> &&
                     common_range<range_reference_t<const V>>)
            return iterator<true>{*this, ranges::end(base_)};
        else
            return sentinel<true>{*this};
        }
    };

    template<class R>
        explicit join_view(R&&) -> join_view<views::all_t<R>>;
}

```

5. Edit 26.7.14.3 [\[range.join.iterator\]](#) as indicated:

```

namespace std::ranges {
    template<input_range V>
        requires view<V> && input_range<range_reference_t<V>>
    template<bool Const>
        struct join_view<V>::iterator {
        private:

```

```

using Parent      = maybe-const<Const, join_view>;           //
    exposition only
using Base        = maybe-const<Const, V>;                   //
    exposition only
using OuterIter   = iterator_t<Base>;                         //
    exposition only
using InnerIter   = iterator_t<range_reference_t<Base>>;       //
    exposition only

static constexpr bool ref-is-glvalue =                       //
    exposition only
    is_reference_v<range_reference_t<Base>>;

OuterIter outer_ = OuterIter();                               //
    exposition only,
+ // present
    only if Base models forward_range
optional<InnerIter> inner_;                                   //
    exposition only
Parent* parent_ = nullptr;                                   //
    exposition only

constexpr void satisfy();                                     //
    exposition only

+ constexpr OuterIter& outer();                               //
    exposition only
+ constexpr const OuterIter& outer() const;                   //
    exposition only

+ constexpr iterator(Parent& parent, OuterIter outer)
+ requires forward_range<Base>;                               //
    exposition only
+ constexpr explicit iterator(Parent& parent)
+ requires (!forward_range<Base>);                             //
    exposition only

public:
    using iterator_concept = see below;
    using iterator_category = see below;                       // not
        always present
    using value_type        = range_value_t<range_reference_t<Base>>;
    using difference_type   = see below;

    iterator() requires default_initializable<OuterIter> = default;
- constexpr iterator(Parent& parent, OuterIter outer);
    constexpr iterator(iterator<!Const> i)
        requires Const &&
            convertible_to<iterator_t<V>, OuterIter> &&
            convertible_to<iterator_t<InnerRng>, InnerIter>;

[...]
```

```

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires ref-is-glvalue && forward_range<Base>equality_ -
        comparable<iterator_t<Base>> &&
        equality_comparable<iterator_t<range_reference_t<Base>>>;

```

```

    [...]
};
}

```

[...]

```

constexpr OuterIter& outer();
constexpr const OuterIter& outer() const;

```

? Returns: *outer_* if *Base* models *forward_range*; otherwise, **parent_ - >outer_*.

```

constexpr void satisfy();

```

5 Effects: Equivalent to:

```

auto update_inner = [this](const iterator_t<Base>& x) -> auto&& {
    if constexpr (ref-is-glvalue)    // *x is a reference
        return *x;
    else
        return parent_ -> inner_.emplace-deref(x);
};
for (; outer()outer_ != ranges::end(parent_ -> base_);
      ++outer()outer_) {
    auto&& inner = update_inner(outer()outer_);
    inner_ = ranges::begin(inner);
    if (inner_ != ranges::end(inner))
        return;
}
if constexpr (ref-is-glvalue)
    inner_.reset();

```

```

constexpr iterator(Parent& parent, OuterIter outer) requires
    forward_range<Base>;

```

6 Effects: Initializes *outer_* with `std::move(outer)` and *parent_* with `addressof(parent)`; then calls *satisfy()*.

```

constexpr explicit iterator(Parent& parent) requires (!forward_range<Base>);

```

? Effects: Initializes *parent_* with `addressof(parent)`; then calls *satisfy()*.

```

constexpr iterator(iterator<!Const> i)
    requires Const &&
        convertible_to<iterator_t<V>, OuterIter> &&
        convertible_to<iterator_t<InnerRng>, InnerIter>;

```

7 Effects: Initializes *outer_* with `std::move(i.outer_)`, *inner_* with `std::move(i.inner_)`, and *parent_* with *i.parent_*.

? [Note ? : Const can only be true when *Base* models *forward_range*. — end note]

```
constexpr InnerIter operator->() const
    requires has_arrow<InnerIter> && copyable<InnerIter>;
```

8 *Effects:* Equivalent to `return inner_;`

```
constexpr iterator& operator++();
```

9 Let *inner-range* be:

(9.1) — If *ref-is-glvalue* is true, `*outer()outer_`.

(9.2) — Otherwise, `*parent_->inner_`.

10 *Effects:* Equivalent to:

```
auto&& inner_rng = inner-range;
if (++inner_ == ranges::end(as-lvalue(inner-range)inner_rng)) {
    ++outer()outer_;
    satisfy();
}
return *this;
```

```
constexpr void operator++(int);
```

11 *Effects:* Equivalent to: `++*this`.

```
constexpr iterator operator++(int)
    requires ref-is-glvalue && forward_range<Base> &&
        forward_range<range_reference_t<Base>>;
```

12 *Effects:* Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

```
constexpr iterator& operator--()
    requires ref-is-glvalue && bidirectional_range<Base> &&
        bidirectional_range<range_reference_t<Base>> &&
        common_range<range_reference_t<Base>>;
```

13 *Effects:* Equivalent to:

```
if (outer_ == ranges::end(parent_->Base_))
    inner_ = ranges::end(as-lvalue(*--outer_));
while (inner_ == ranges::begin(as-lvalue(*outer_)))
    inner_ = ranges::end(as-lvalue(*--outer_));
--inner_;
return *this;
```

```
constexpr iterator operator--(int)
    requires ref-is-glvalue && bidirectional_range<Base> &&
        bidirectional_range<range_reference_t<Base>> &&
        common_range<range_reference_t<Base>>;
```

14 *Effects:* Equivalent to:

```

    auto tmp = *this;
    --*this;
    return tmp;

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires ref-is-glvalue && forward_range<Base>equality_-
        comparable<iterator_t<Base>> &&
        equality_comparable<iterator_t<range_reference_t<Base>>>;

15 Effects: Equivalent to: return x.outer_ == y.outer_ && x.inner_ ==
    y.inner_;

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(x.inner_, y.inner_)))
    requires indirectly_swappable<InnerIter>;

16 Effects: Equivalent to: return ranges::iter_swap(x.inner_, y.inner_);

```

6. Edit 26.7.14.4 [\[range.join.sentinel\]](#) p3 as indicated:

```

template<bool OtherConst>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst,
        V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const
    sentinel& y);

3 Effects: Equivalent to: return x.outer().outer_ == y.end_;

```

7. Edit 26.7.15.2 [\[range.join.with.view\]](#), class template join_with_view synopsis, as indicated:

```

namespace std::ranges {
    [...]

    template<input_range V, forward_range Pattern>
        requires view<V> && input_range<range_reference_t<V>>
            && view<Pattern>
            && compatible_joinable_ranges<range_reference_t<V>, Pattern>
    class join_with_view : public view_interface<join_with_view<V, Pattern>>
    {
        using InnerRng = range_reference_t<V>; // exposition
        only

        V base_ = V(); // exposition
        only
+ non-propagating-cache<iterator_t<V>> outer_it_; // exposition
        only, present only
+ // when
        !forward_range<V>
        non-propagating-cache<remove_cv_t<InnerRng>> inner_; // exposition
        only, present only
        // when
        !is_reference_v<InnerRng>
        Pattern pattern_ = Pattern(); // exposition
        only
    };

```



```

// 26.7.15.3 \[range.join.with.iterator\], class template
join_with_view::iterator
template<bool Const> struct iterator; // exposition
only

// 26.7.15.4 \[range.join.with.sentinel\], class template
join_with_view::sentinel
template<bool Const> struct sentinel; // exposition
only

public:
    [...]

    constexpr auto begin() {
+   if constexpr (forward_range<V>) {
        constexpr bool use_const =
            simple-view<V> && is_reference_v<InnerRng> && simple-
            view<Pattern>;
        return iterator<use_const>{*this, ranges::begin(base_)};
+   }
+   else {
+       outer_it_ = ranges::begin(base_);
+       return iterator<false>{*this};
+   }
    }

    constexpr auto begin() const
        requires forward_input_range<const V> &&
            forward_range<const Pattern> &&
            is_reference_v<range_reference_t<const V>> &&
+       input_range<range_reference_t<const V>>
    { return iterator<true>{*this, ranges::begin(base_)}; }

    constexpr auto end() {
        if constexpr (forward_range<V> &&
            is_reference_v<InnerRng> && forward_range<InnerRng> &&
            common_range<V> && common_range<InnerRng>)
            return iterator<simple-view<V> && simple-view<Pattern>>{*this,
                ranges::end(base_)};
        else
            return sentinel<simple-view<V> && simple-view<Pattern>>{*this};
    }

    constexpr auto end() const
        requires forward_input_range<const V> && forward_range<const Pattern>
            &&
            is_reference_v<range_reference_t<const V>> &&
+       input_range<range_reference_t<const V>>
    {
        using InnerConstRng = range_reference_t<const V>;
        if constexpr (forward_range<const V> && forward_range<InnerConstRng>
            &&
            common_range<const V> && common_range<InnerConstRng>)
            return iterator<true>{*this, ranges::end(base_)};
        else
            return sentinel<true>{*this};
    }
}

```

```
};

[...]
```

8. Edit 26.7.15.3 [\[range.join.with.iterator\]](#) as indicated:

```
namespace std::ranges {
    template<input_range V, forward_range Pattern>
        requires view<V> && input_range<range_reference_t<V>>
            && view<Pattern> && compatible-joinable-
                ranges<range_reference_t<V>, Pattern>
    template<bool Const>
    class join_with_view<V, Pattern>::iterator {
        using Parent = maybe-const<Const, join_with_view>; //
        exposition only
        using Base = maybe-const<Const, V>; //
        exposition only
        using InnerBase = range_reference_t<Base>; //
        exposition only
        using PatternBase = maybe-const<Const, Pattern>; //
        exposition only

        using OuterIter = iterator_t<Base>; //
        exposition only
        using InnerIter = iterator_t<InnerBase>; //
        exposition only
        using PatternIter = iterator_t<PatternBase>; //
        exposition only

        static constexpr bool ref-is-glvalue = is_reference_v<InnerBase>; //
        exposition only

        Parent* parent_ = nullptr; //
        exposition only
        OuterIter outer_it_ = OuterIter(); //
        exposition only,
+ //
        present only if Base models forward_range
        variant<PatternIter, InnerIter> inner_it_; //
        exposition only

-     constexpr iterator(Parent& parent, iterator_t<Base> outer); //
        exposition only
+     constexpr iterator(Parent& parent, OuterIter outer)
+     requires forward_range<Base>; //
        exposition only
+     constexpr explicit iterator(Parent& parent)
+     requires (!forward_range<Base>); //
        exposition only
+     constexpr OuterIter& outer(); //
        exposition only
+     constexpr const OuterIter& outer() const; //
        exposition only
        constexpr auto&& update-inner(const OuterIter&); //
        exposition only
        constexpr auto&& get-inner(const OuterIter&); //
        exposition only
```

```

constexpr void satisfy(); //
    exposition only

public:
    using iterator_concept = see below;
    using iterator_category = see below; //
        not always present
    using value_type = see below;
    using difference_type = see below;

    iterator() requires default_initializable<OuterIter> = default;
    constexpr iterator(iterator<!Const> i)
        requires Const && convertible_to<iterator_t<V>, OuterIter> &&
            convertible_to<iterator_t<InnerRng>, InnerIter> &&
            convertible_to<iterator_t<Pattern>, PatternIter>;

    [...]

    friend constexpr bool operator==(const iterator& x, const iterator& y)
        requires ref-is-glvalue && forward_range<Base>equality_
            comparable<OuterIter> &&
            equality_comparable<InnerIter>;

    [...]
};
}

```

[...]

```

constexpr OuterIter& outer();
constexpr const OuterIter& outer() const;

```

? Returns: *outer_it_* if *Base* models *forward_range*; otherwise, **parent_* - *>outer_it_*.

```
constexpr auto&& update-inner(const OuterIter& x);
```

5 Effects: Equivalent to:

```

if constexpr (ref-is-glvalue)
    return as-lvalue(*outer()x);
else
    return parent_ -> inner_.emplace-deref(outer()x);

```

```
constexpr auto&& get-inner(const OuterIter& x);
```

6 Effects: Equivalent to:

```

if constexpr (ref-is-glvalue)
    return as-lvalue(*outer()x);
else
    return *parent_ -> inner_;

```

```
constexpr void satisfy();
```

5 *Effects:* Equivalent to:

```
while (true) {
    if (inner_it_.index() == 0) {
        if (std::get<0>(inner_it_) != ranges::end(parent_>pattern_))
            break;
        auto&& inner = update-inner(outer_it_);
        inner_it_.emplace<1>(ranges::begin(update-inner()inner));
    } else {
        auto&& inner = get-inner(outer_it_);
        if (std::get<1>(inner_it_) != ranges::end(get-inner()inner))
            break;
        if (++outer()outer_it_ == ranges::end(parent_>base_)) {
            if constexpr (ref-is-glvalue)
                inner_it_.emplace<0>();
            break;
        }
        inner_it_.emplace<0>(ranges::begin(parent_>pattern_));
    }
}
```

[Note 1: join_with_view iterators use the *satisfy* function to skip over empty inner ranges. — end note]

```
constexpr iterator(Parent& parent, OuterIteriterator_t<Base> outer) requires
    forward_range<Base>;
constexpr explicit iterator(Parent& parent) requires (!forward_range<Base>;
```

6 *Effects:* Initializes *parent_* with `addressof(parent)`. For the first overload, also initializes and *outer_it_* with `std::move(outer)`. Then, equivalent to:

```
if (outer()outer_it_ != ranges::end(parent_>base_)) {
    auto&& inner = update-inner(outer_it_);
    inner_it_.emplace<1>(ranges::begin(update-inner()inner));
    satisfy();
}
```

```
constexpr iterator(iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, OuterIter> &&
        convertible_to<iterator_t<InnerRng>, InnerIter> &&
        convertible_to<iterator_t<Pattern>, PatternIter>;
```

7 *Effects:* Initializes *outer_it_* with `std::move(i.outer_it_)` and *parent_* with *i.parent_*. Then, equivalent to:

```
if (i.inner_it_.index() == 0)
    inner_it_.emplace<0>(std::get<0>(std::move(i.inner_it_)));
else
    inner_it_.emplace<1>(std::get<1>(std::move(i.inner_it_)));
```

? [Note ?: Const can only be true when *Base* models *forward_range*. — end note]

[...]

[Drafting note: The specification of operator `--` in paragraph 14 can be editorially simplified using the new *as-lvalue* helper; that cleanup is not included in this paper as there is no normative impact.]

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires ref-is-glvalue && forward_range<Base>equality_ -
        comparable<OuterIter> &&
        equality_comparable<iterator_t<range_reference_t<Base>>>;
```

16 *Effects:* Equivalent to: `return x.outer_it_ == y.outer_it_ && x.inner_`
`== y.inner_;`

9. Edit 26.7.15.4 [\[range.join.with.sentinel\]](#) p3 as indicated:

```
template<bool OtherConst>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst,
        V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const
    sentinel& y);
```

3 *Effects:* Equivalent to: `return x.outer().outer_it_ == y.end_;`

§ 4 References

[LWG3698] Barry Revzin. `regex_iterator` and `join_view` don't work together very well.

<https://wg21.link/lwg3698>

[LWG3700] Hewill Kang. The `const` begin of the `join_view` family does not require `InnerRng` to be a range.

<https://wg21.link/lwg3700>

[LWG3791] Hewill Kang. `join_view::iterator::operator-` may be ill-formed.

<https://wg21.link/lwg3791>

[N4928] Thomas Köppe. 2022-12-18. Working Draft, Standard for Programming Language C++.

<https://wg21.link/n4928>